



viettel

Theo cách của bạn



VIETTEL DIGITAL TALENT 2026

Streaming Lakehouse Data Platform

Real-time data pipelines with Apache Kafka & Apache Spark Structured Streaming

Mentor: Nguyen Minh Hieu

Student: Ngo Quang Hieu

Data Engineer Track
Viettel Digital Talent 2026

July 1st, 2026

Table of Contents

- 1 Introduction
- 2 Background & Design Choices
- 3 Problem & Challenges
- 4 Proposed Solution
- 5 Message Queue: Apache Kafka
- 6 Stream Processing: Spark Structured Streaming & Hudi
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset
- 8 Monitoring & Evaluation
- 9 Conclusion
- 10 References

Table of Contents

- 1 Introduction**
- 2 Background & Design Choices
- 3 Problem & Challenges
- 4 Proposed Solution
- 5 Message Queue: Apache Kafka
- 6 Stream Processing: Spark Structured Streaming & Hudi
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset
- 8 Monitoring & Evaluation
- 9 Conclusion
- 10 References

Why Stream Processing?

The Problem with Batch

- **High latency** — jobs run hourly/daily, insights lag by hours
- **Stale decisions** — by the time data is ready, the moment to act has passed
- **Resource spikes** — large periodic loads strain infra, then sit idle

Batch: hours of delay



Stream: milliseconds

Stream Processing Enables

- **Live monitoring & alerting** — detect anomalies as they happen
- **Continuous aggregation** — dashboards show current metrics, not a stale snapshot
- **Smooth, steady load** — events processed as they arrive, no periodic resource spikes

Objectives

Objectives

- ➊ Overview **Apache Kafka** as a distributed message queue
- ➋ Overview **Apache Spark Structured Streaming** as a real-time stream processing engine
- ➌ Build an end-to-end **ELT streaming pipeline** — from event simulation, through a **Hudi lakehouse**, to a **Superset** BI dashboard
- ➍ Demonstrate **observability** with Prometheus & Grafana monitoring

Table of Contents

- 1 Introduction
- 2 Background & Design Choices**
- 3 Problem & Challenges
- 4 Proposed Solution
- 5 Message Queue: Apache Kafka
- 6 Stream Processing: Spark Structured Streaming & Hudi
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset
- 8 Monitoring & Evaluation
- 9 Conclusion
- 10 References

Kafka: Publish/Subscribe

- Producers write to **topics**; consumers subscribe independently
- Durable, replicated, append-only log — replay any offset

Structured Streaming: Micro-Batch

- Treats a stream as an unbounded, incrementally-processed table
- Each trigger reads a bounded slice of new Kafka records and runs a Spark batch job over it

Hudi: COW vs. MOR

- **Copy-on-Write (COW)** — rewrites base files on every write; fast reads, simple
- **Merge-on-Read (MOR)** — appends to delta log files, compacted later; fast writes for high-frequency streaming appends

Why Spark + Hudi + Superset?

- **Spark Structured Streaming vs. Flink** — one engine (Spark) covers both the streaming ingest job and the batch SQL that dbt submits, so the same 4-core cluster serves both without running two separate engines
- **Hudi vs. Snowflake** — an open table format on commodity object storage (MinIO/S3A); no managed warehouse subscription, and MOR bronze tables absorb frequent small streaming writes cheaply
- **Superset vs. Power BI** — open-source, connects directly to the same Spark Thrift Server the pipeline already runs, no extra licensing or export step

Table of Contents

- 1 Introduction
- 2 Background & Design Choices
- 3 Problem & Challenges**
- 4 Proposed Solution
- 5 Message Queue: Apache Kafka
- 6 Stream Processing: Spark Structured Streaming & Hudi
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset
- 8 Monitoring & Evaluation
- 9 Conclusion
- 10 References

Eventsim

- Simulates page requests for a fake music-streaming website — synthetic but realistic user behaviour
- Generates three event streams: `listen_events`, `page_view_events`, `auth_events`

Million Song Dataset (subset)

- Eventsim draws song/artist metadata from a **10,000-song subset** of the Million Song Dataset
- Gives listen events realistic song, artist, and duration attributes

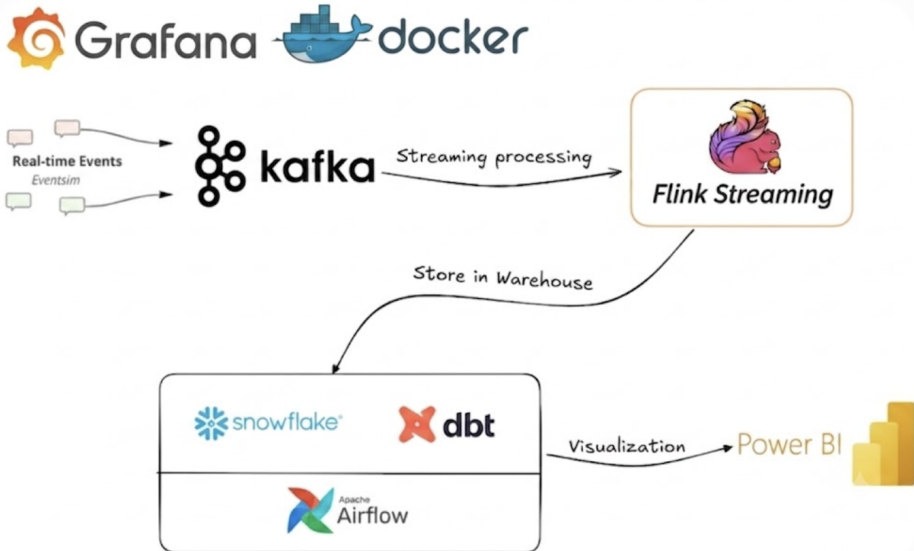
Technical Challenges

- **Continuous streaming volume** — events must be ingested, decoded, and enriched (mojibake fixes, derived time columns) without falling behind Kafka
- **Small, shared compute** — the whole Spark Standalone cluster is **1 master + 2 workers, 4 cores total**, split between the streaming job and the dbt Thrift Server, with no headroom for ad-hoc SQL
- **Partitioning & parallelism** — Kafka topics have only 3 partitions each; partition count, replication factor, and Spark's capped cores all bound end-to-end throughput

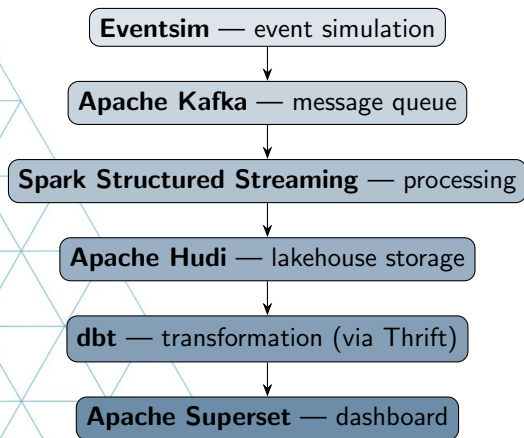
Table of Contents

- 1 Introduction
- 2 Background & Design Choices
- 3 Problem & Challenges
- 4 Proposed Solution**
- 5 Message Queue: Apache Kafka
- 6 Stream Processing: Spark Structured Streaming & Hudi
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset
- 8 Monitoring & Evaluation
- 9 Conclusion
- 10 References

Streamify: End-to-End Architecture



Architecture: Layer by Layer



3 Event Streams Simulated

- `listen_events` — song plays, skips, durations
- `page_view_events` — user navigation behaviour
- `auth_events` — logins and session events

Deployment

- Fully containerised with **Docker Compose**
- Single command: `docker compose up`
- Spark Standalone: **1 master + 2 workers** (2 cores / 2 GB each), shared by the streaming job and the dbt Thrift Server

- ❶ **Kafka** — Eventsim publishes to `listen_events`, `page_view_events`, `auth_events`
- ❷ **Spark Structured Streaming** — reads all three topics, fixes latin1-mojibake, derives time columns, appends each micro-batch to **bronze** Hudi tables (`staging.<topic>`, `MERGE_ON_READ`)
- ❸ **dbt (dbt-spark, Thrift)** — builds the star schema (`fact_streams` + 5 dims) and the `wide_streams` table on top of bronze, materialized as `COPY_ON_WRITE` Hudi tables
- ❹ **Superset** — dashboards query `wide_streams` through the same Spark Thrift Server

Table of Contents

- 1 Introduction
- 2 Background & Design Choices
- 3 Problem & Challenges
- 4 Proposed Solution
- 5 Message Queue: Apache Kafka**
- 6 Stream Processing: Spark Structured Streaming & Hudi
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset
- 8 Monitoring & Evaluation
- 9 Conclusion
- 10 References

Why Apache Kafka?

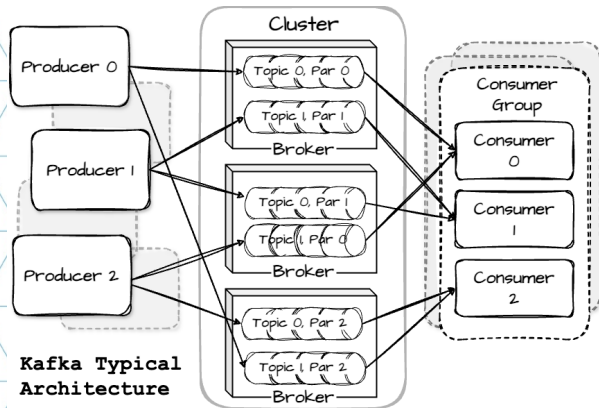
Problems Without a Message Queue

- **Tight coupling** — producer knows every consumer
- **No buffering** — slow consumer → data loss
- **No replay** — consumed = gone forever
- **Fan-out cost** — N consumers = N direct calls

How Kafka Solves It

- **Decoupling** — write to topic, subscribe independently
- **Durable buffer** — persisted to disk, replicated
- **Replay** — configurable retention, re-read anytime
- **Scalability** — partitions scale throughput linearly
- **Fan-out free** — multiple groups, own offsets

Kafka Architecture



- **Producer** — partitions by key; acks=all for durability
- **Broker** — leader/follower replicas; append-only log
- **Topic** — immutable records; retention-based or compacted
- **Partition** — ordered, offset-indexed; unit of parallelism
- **Consumer Group** — one partition → one member; independent offset per group

Max parallelism = number of partitions

Kafka Setup in Streamify

2-Node KRaft Cluster

- Runs on **apache/kafka:4.3.0** — no Zookeeper
- Each node (broker-1, broker-2) is both broker & controller
- **Replication factor 2** — every partition has a replica on both brokers, no data loss if one broker restarts

Topic Configuration

- 3 topics: `listen_events`, `page_view_events`, `auth_events`
- **3 partitions** per topic — enables parallel consumption by Spark Structured Streaming
- Topics pre-created by `kafka-init` before producers start

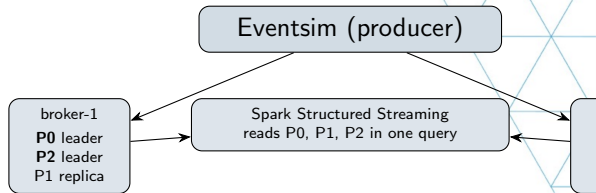
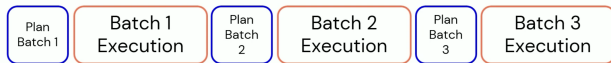


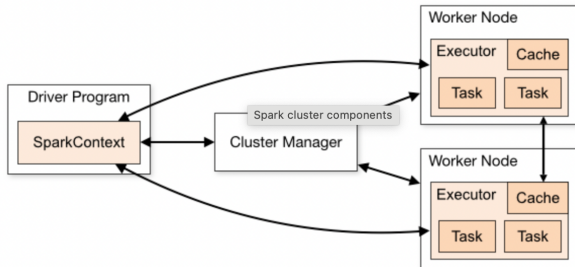
Table of Contents

- 1 Introduction
- 2 Background & Design Choices
- 3 Problem & Challenges
- 4 Proposed Solution
- 5 Message Queue: Apache Kafka
- 6 Stream Processing: Spark Structured Streaming & Hudi**
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset
- 8 Monitoring & Evaluation
- 9 Conclusion
- 10 References

Spark Structured Streaming: Micro-Batch Model



Each trigger processes a bounded batch of new Kafka records as a Spark job



Dataflow Model

- **Micro-batch** — the stream is an unbounded table, incrementally queried on a trigger interval
- Same Spark engine and SQL/DataFrame API as batch jobs
- Result: each batch's rows appended straight to Hudi

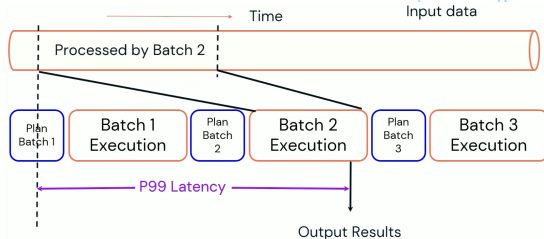
Fault Tolerance

- Structured Streaming **checkpoint** tracks processed Kafka offsets and query state
- Persisted to the spark-checkpoints Docker volume — resumes from last committed offset on restart
- Offsets also committed to a

Writing to Hudi: Bronze Layer

MERGE_ON_READ Bronze Tables

- Each micro-batch appends to staging.<topic> as a Hudi **MERGE_ON_READ (MOR)** table
- Fast streaming appends land as delta log files, periodically compacted into columnar base files
- Backed by MinIO (dev) or S3 (aws mode) via the Hadoop S3A connector — no rebuild needed to switch
- No separate catalog service — each table self-describes via its .hoodie/ metadata directory, (re-)registered into Spark's catalog at job startup



Spark Setup in Streamify

Shared Standalone Cluster

- 1 master + 2 workers, 2 cores / 2 GB each — 4 cores total
- **spark-streaming** job (Kafka → Hudi bronze), capped at 2 cores
- **spark-thrift** Thrift Server (dbt SQL submission), capped at 2 cores
- Full cluster used by the two long-running applications — no headroom for ad-hoc Spark SQL at this size

Streaming Pipeline

- Source: Kafka connector (3 topics)
- Transform: fix latin1-mojibake on text fields, derive time columns
- Sink: Hudi bronze tables, MERGE_ON_READ

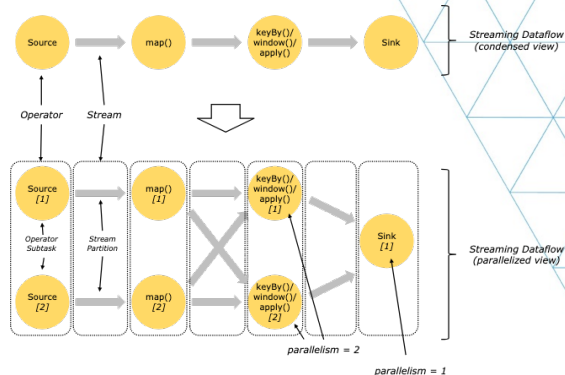
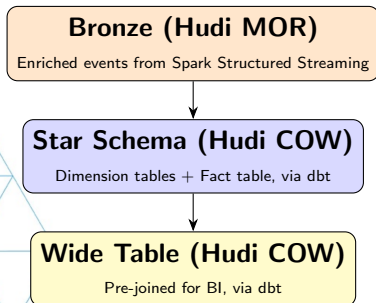


Table of Contents

- 1 Introduction
- 2 Background & Design Choices
- 3 Problem & Challenges
- 4 Proposed Solution
- 5 Message Queue: Apache Kafka
- 6 Stream Processing: Spark Structured Streaming & Hudi
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset**
- 8 Monitoring & Evaluation
- 9 Conclusion
- 10 References

Lakehouse: Bronze → Star Schema → Wide Table



Bronze — Streaming Ingest

- staging.<topic> tables, MERGE_ON_READ, written directly by the Spark streaming job

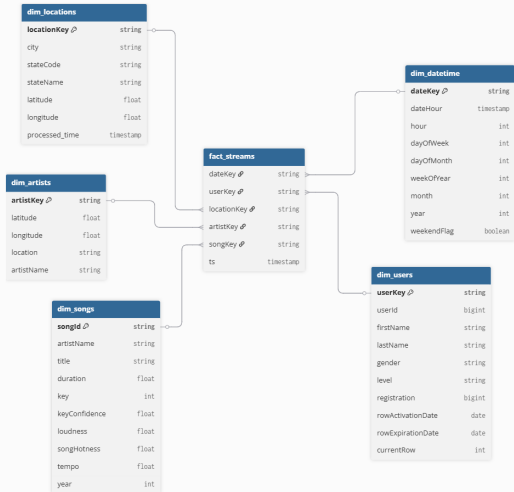
Star Schema — dbt (Thrift)

- fact_streams central fact table
- 5 dimension tables: users, songs, artists, locations, datetime
- Materialized as COPY_ON_WRITE Hudi tables, rebuilt via full overwrite each run

Wide Table — dbt (Thrift)

- wide_streams: pre-joins fact + all dims
- Single flat table for Superset — no runtime JOINS

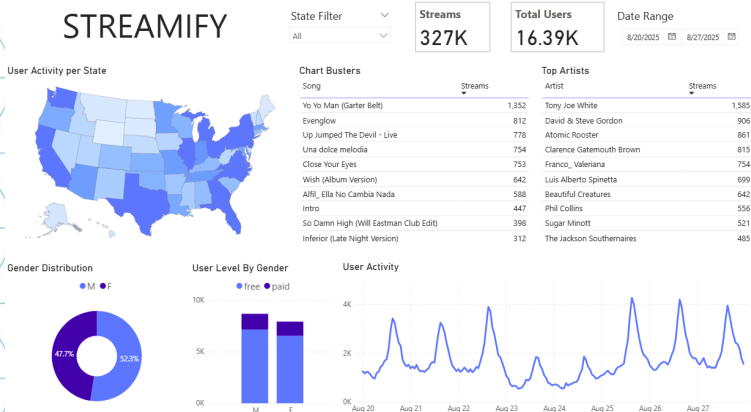
Star Schema & dbt DAG



dbdiagram.io



Dashboard: Real-time Insights



Key Metrics (Superset)

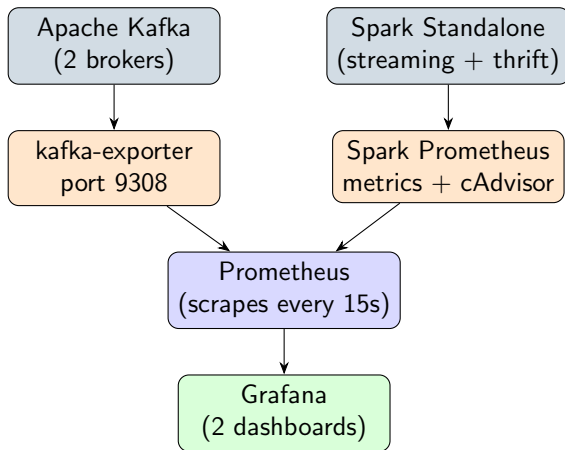
- **Top artists** by total play count
- **Songs played over time** — hourly trend
- **Geographic distribution** — listener locations (US map)
- **Free vs. paid** listener split
- **Session activity** — active users over time

Connected through the same **Spark Thrift Server** that dbt uses, querying `wide_streams` directly.

Table of Contents

- 1 Introduction
- 2 Background & Design Choices
- 3 Problem & Challenges
- 4 Proposed Solution
- 5 Message Queue: Apache Kafka
- 6 Stream Processing: Spark Structured Streaming & Hudi
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset
- 8 Monitoring & Evaluation**
- 9 Conclusion
- 10 References

Monitoring: How It Fits Together



Monitoring Stack: Prometheus & Grafana

Why Monitor?

- Verify Spark streaming throughput matches Kafka ingestion rate
- Detect consumer-group lag
- Track CPU and memory across the 4-core shared Spark cluster (via cAdvisor)

Components

- **Spark's built-in Prometheus metrics** — JVM heap, streaming batch duration/throughput
- **kafka-exporter** — exposes consumer-group lag on port 9308
- **cAdvisor** — per-container CPU/memory
- **Prometheus** — scrapes all of it; stores time-series data

- **Grafana** — visualises with 2 pre-built

2 Grafana Dashboards

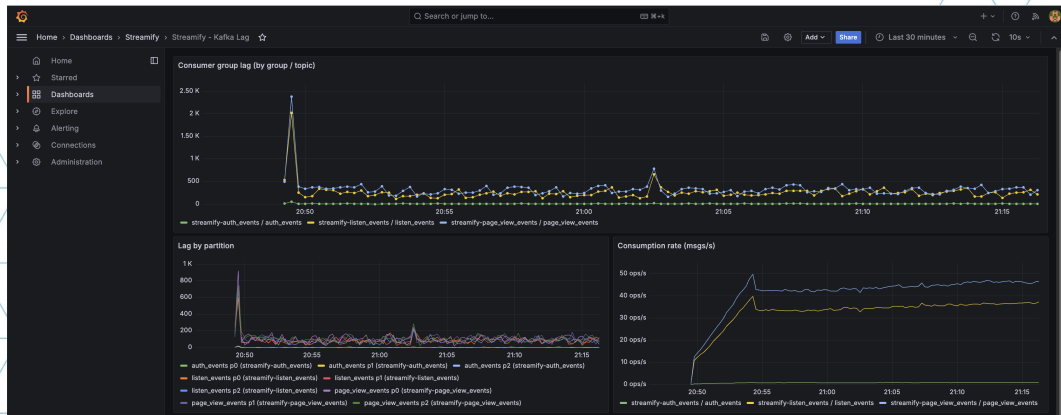
Kafka Lag (kafka-lag.json)

Consumer-group lag per topic + consumption rate (msgs/s) — detects if Spark streaming is falling behind

Spark (spark.json)

Cluster health, JVM heap, streaming throughput/latency per topic, worker resource usage

Dashboard: Kafka Consumer Lag



Metric: `kafka_consumergroup_lag` — consumer-group offset behind the latest partition offset. A lag of 0 means Spark Structured Streaming is processing events as fast as Kafka receives them.

Dashboard: Spark Monitoring



Metrics: cluster health, JVM heap, streaming throughput/latency per Kafka topic, per-worker CPU/memory usage.

Evaluation: Test Approach

Test Environment

- Full stack via docker compose up — Kafka (2 brokers), Spark Standalone (1 master + 2 workers), dbt, Superset, Prometheus/Grafana
- Eventsim generating synthetic traffic against the 10,000-song Million Song Dataset subset

Correctness & Runtime Checks

- Grafana Kafka Lag dashboard confirms the streaming job keeps consumer-group lag near zero
- Grafana Spark dashboard confirms stable JVM heap and per-batch throughput over time
- dbt run/test confirms star-schema and wide-table models build successfully on top of bronze

TODO

No benchmark numbers (throughput, latency percentiles, resource utilisation figures) are recorded in this repository yet — add them here once a dedicated load test is run.

Table of Contents

- 1 Introduction
- 2 Background & Design Choices
- 3 Problem & Challenges
- 4 Proposed Solution
- 5 Message Queue: Apache Kafka
- 6 Stream Processing: Spark Structured Streaming & Hudi
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset
- 8 Monitoring & Evaluation
- 9 Conclusion**
- 10 References

Conclusion

Results Achieved

- 1 **Kafka** decouples producers from consumers — 2-node KRaft cluster, RF2, 3 partitions per topic gives fault tolerance without a third broker
- 2 **Spark Structured Streaming** ingests all 3 topics as micro-batches, enriches them, and appends to **Hudi MERGE_ON_READ** bronze tables, resuming from a persisted checkpoint on restart
- 3 **ELT pattern** keeps concerns separate: Spark loads raw events into Hudi, dbt transforms in place into a star schema and wide table via the Spark Thrift Server
- 4 **Superset** dashboards and **Prometheus/Grafana** monitoring close the loop — data is queryable and the pipeline's health is visible in real time

Limitations

- The 4-core Spark Standalone cluster is shared between streaming ingest and the dbt Thrift Server, leaving no headroom for ad-hoc SQL
- Gold-layer dbt models rebuild via full overwrite rather than incremental merge
- No recorded throughput/latency benchmarks yet (see Evaluation)

Table of Contents

- 1 Introduction
- 2 Background & Design Choices
- 3 Problem & Challenges
- 4 Proposed Solution
- 5 Message Queue: Apache Kafka
- 6 Stream Processing: Spark Structured Streaming & Hudi
- 7 Lakehouse & Dashboard: Hudi, dbt, Superset
- 8 Monitoring & Evaluation
- 9 Conclusion
- 10 References**

References

- Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- Spark Structured Streaming Programming Guide. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- Apache Hudi Documentation. <https://hudi.apache.org/docs/overview>
- dbt Documentation (dbt-spark adapter).
<https://docs.getdbt.com/docs/core/connect-data-platform/spark-setup>
- Apache Superset Documentation. <https://superset.apache.org/docs/intro>
- Eventsim (viirya's fork). <https://github.com/viirya/eventsim>
- Million Song Dataset. <http://millionsongdataset.com>

- THE END -

Thank you for your attention

Contact:

Ngo Quang Hieu — Viettel Digital Talent 2026